

Informe de Análisis Estático de Código (Linter)

Proyecto: Calculadora de Porciones e Inyecciones de Hornos

Estado: Certificado con Cero Errores y Advertencias

Fecha: Junio de 2026

1. Herramienta Utilizada

Se ha seleccionado **ESLint v9.x** como motor de análisis estático, en combinación con **typescript-eslint** para dar soporte oficial y tipado estricto sobre el ecosistema de TypeScript 5 y Next.js.

- **Linter Principal:** eslint
- **Analizador Sintáctico (Parser):** @typescript-eslint/parser
- **Formato de Configuración:** Flat Config (eslint.config.js)

2. Descripción de las Reglas de Calidad Aplicadas

En el archivo de configuración `eslint.config.js` se ha definido un conjunto de reglas personalizadas diseñadas para mantener la cohesión estilística, prevenir malas prácticas y evitar errores silenciosos en tiempo de ejecución. A continuación, se detalla el comportamiento y propósito de cada una de ellas:

A. Consistencia y Formato Estilístico

- **indent (Tabulaciones obligatorias):** Configurada con el valor `["error", "tab", { "SwitchCase": 1 }]`. Fuerza el uso de tabulaciones físicas (*Tabs*) en lugar de espacios para estructurar la jerarquía visual del código. La subregla `SwitchCase: 1` garantiza que los bloques `case` y `default` dentro de una estructura `switch` se indenten un nivel por delante del bloque principal para mejorar la legibilidad del flujo de control.
- **quotes (Comillas dobles):** Definida como `["error", "double", { "avoidEscape": true, "allowTemplateLiterals": true }]`. Exige que todos los literales de cadena utilicen comillas dobles ("x"). Sin embargo, contempla dos excepciones cruciales:
 - `avoidEscape: true`: Permite usar comillas simples si la cadena ya contiene comillas dobles en su interior, evitando tener que escaparlas con barras invertidas (`\`).
 - `allowTemplateLiterals: true`: Permite utilizar acentos graves (```) para crear plantillas literales que requieran interpolación de variables.

- **semi (Uso de punto y coma):** Configurada como `["error", "always"]`. Hace obligatorio el uso de punto y coma `;` al finalizar sentencias y declaraciones. Esto evita que el motor de JavaScript realice la "Inserción Automática de Puntos y Comas".
- **eol-last (Salto de línea al final del archivo):** Establecida en `["error", "always"]`. Exige que todos los archivos finalicen con exactamente una línea en blanco

B. Nomenclatura y Convención de Código

- **camelcase (Nomenclatura camelCase obligatoria):** Configurada como `["error", { "properties": "always", "ignoreDestructuring": false }]`. Garantiza que todos los nombres de variables, funciones y constantes sigan el estilo camelCase (ejemplo: `calcHandlingTime`). La propiedad `properties: "always"` fuerza que las propiedades declaradas en objetos también lo cumplan, mientras que `ignoreDestructuring: false` asegura que el mismo rigor se aplique a las variables extraídas por desestructuración de datos.

C. Buenas Prácticas y Limpieza de Variables

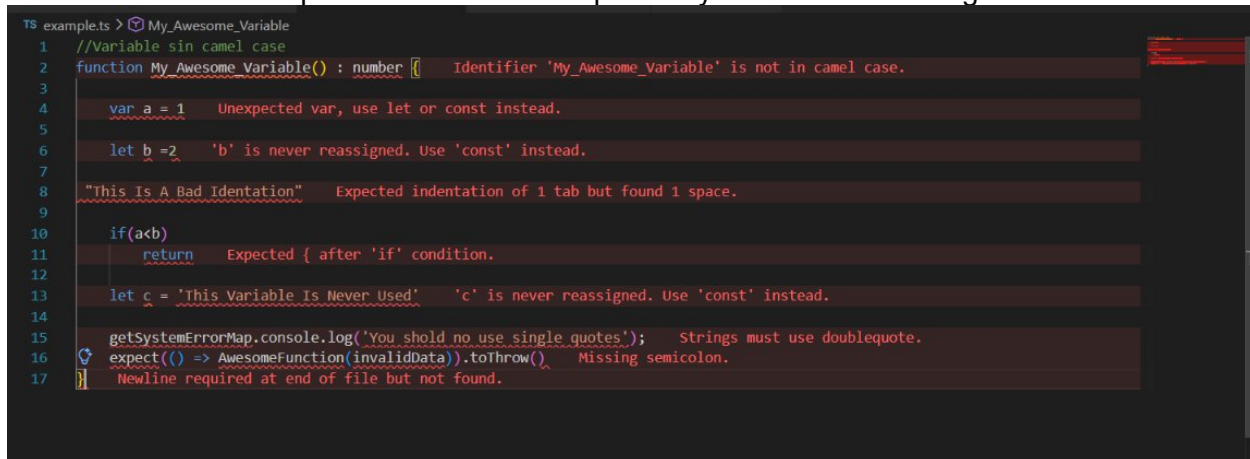
- **no-unused-vars / @typescript-eslint/no-unused-vars (Variables no utilizadas):** Para evitar conflictos con el tipado de TypeScript, la regla nativa de JavaScript `no-unused-vars` se ha apagado ("off"), delegando la responsabilidad en la versión especializada de TypeScript: `@typescript-eslint/no-unused-vars`. Configurada como `["warn", { "argsIgnorePattern": "^_", "varsIgnorePattern": "^_" }]`, emite advertencias ante la presencia de variables o argumentos declarados que no se consumen. Permite, de manera excepcional, omitir la advertencia si el nombre del argumento o de la variable inicia con un guion bajo (por ejemplo: `_unusedVar`), lo cual es útil para callbacks o firmas de funciones requeridas por librerías de terceros.
- **prefer-const (Uso preferente de constantes):** Definida en nivel "error". Identifica todas las variables declaradas con `let` que nunca vuelven a ser reasignadas tras su inicialización, obligando al desarrollador a redeclararlas bajo la palabra clave `const`. Esto optimiza la inmutabilidad y mejora el razonamiento del código.
- **no-var (Prohibición de var):** Establecida en nivel "error". Prohíbe estrictamente el uso de la palabra clave tradicional `var`. Obliga a los desarrolladores a utilizar `let` para variables mutables con ámbito de bloque, o `const` para valores constantes, mitigando errores relacionados con el *hoisting* o fugas de variables.

- **curly (Uso obligatorio de llaves):** Configurada como `["error", "all"]`. Obliga a que todas las estructuras de control (`if`, `else`, `for`, `while`, `do`) utilicen llaves `{}` para delimitar sus bloques de ejecución, incluso si contienen una única instrucción. De este modo, se previenen errores comunes al añadir líneas adicionales a bloques condicionales que previamente omitían las llaves.

3. Ejecución y prueba

Entorno controlado

Se creó un archivo de pruebas donde se rompió una y cada una de las reglas



```

TS example.ts > My_Awesome_Variable
1 //Variable sin camel case
2 function My_Awesome_Variable() : number { Identifier 'My_Awesome_Variable' is not in camel case.
3
4   var a = 1 Unexpected var, use let or const instead.
5
6   let b = 2 'b' is never reassigned. Use 'const' instead.
7
8   "This Is A Bad Indentation" Expected indentation of 1 tab but found 1 space.
9
10  if(a<b)
11    return Expected { after 'if' condition.
12
13  let c = 'This Variable Is Never Used' 'c' is never reassigned. Use 'const' instead.
14
15  getSystemErrorMap.console.log('You shold no use single quotes'); Strings must use doublequote.
16  expect(() => AwesomeFunction(invalidData)).toThrow() Missing semicolon.
17 } Newline required at end of file but not found.
  
```

Se procedió con `npm eslint`.

`npx eslint`.

`C:\**\Desktop\New folder (2)\project-backend\example.ts`

2:10 error Identifier 'My_Awesome_Variable' is not in camel case

camelcase

2:10 warning 'My_Awesome_Variable' is defined but never used. Allowed unused vars must match `/^_u` @typescript-eslint/no-unused-vars

4:2 error Unexpected var, use let or const instead

no-var

4:11 error Missing semicolon

semi

6:6 error 'b' is never reassigned. Use 'const' instead

prefer-const

6:10 error Missing semicolon

semi

8:1 error Expected indentation of 1 tab but found 1 space

indent

8:2 error Expected an assignment or function call and instead saw an expression

@typescript-eslint/no-unused-expressions

8:28 error Missing semicolon

semi

11:3 error Expected { after 'if' condition

curly

11:9 error Missing semicolon

semi

13:6 error 'c' is never reassigned. Use 'const' instead

prefer-

```

const
13:6  warning  'c' is assigned a value but never used. Allowed unused vars must match /^_/u
@typescript-eslint/no-unused-vars
13:10  error    Strings must use doublequote                quotes
13:39  error    Missing semicolon                            semi
15:32  error    Strings must use doublequote                quotes
16:54  error    Missing semicolon                            semi
17:2   error    Newline required at end of file but not found    eol-last

```

✖ 18 problems (16 errors, 2 warnings)

14 errors and 0 warnings potentially fixable with the `--fix` option.

Reparación automática

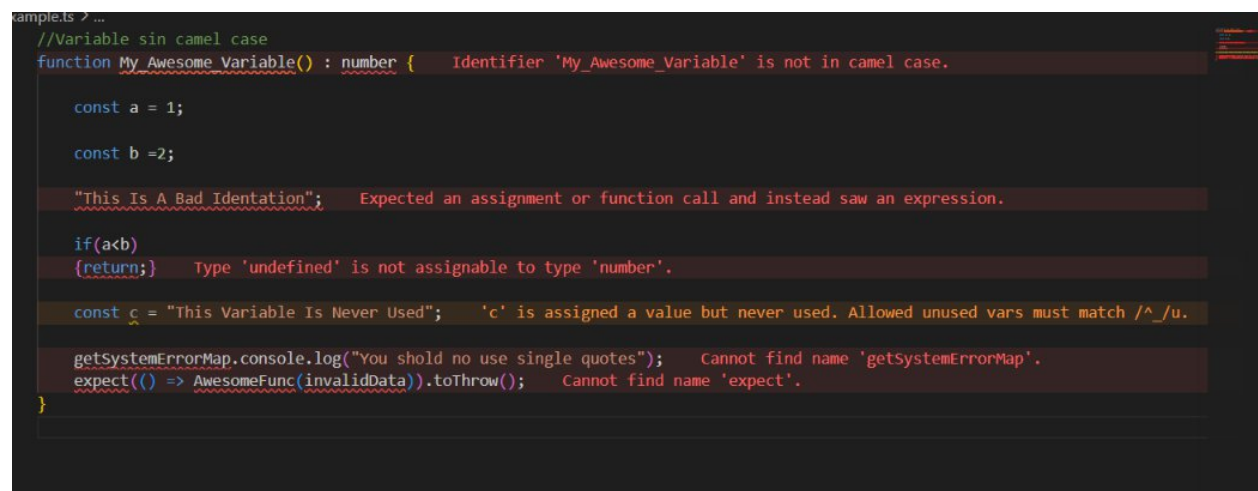
PS C:\Users\Nikor\Desktop\New folder (2)\project-backend> npx eslint --fix

```

C:\**\**\Desktop\New folder (2)\project-backend\example.ts
2:10  error  Identifier 'My_Awesome_Variable' is not in camel case
camelcase
2:10  warning  'My_Awesome_Variable' is defined but never used. Allowed unused vars must
match /^_/u @typescript-eslint/no-unused-vars
8:2   error  Expected an assignment or function call and instead saw an expression
@typescript-eslint/no-unused-expressions
13:8  warning  'c' is assigned a value but never used. Allowed unused vars must match /^_/u
@typescript-eslint/no-unused-vars

```

✖ 4 problems (2 errors, 2 warnings)



```

//Variable sin camel case
function My_Awesome_Variable() : number { Identifier 'My_Awesome_Variable' is not in camel case.

    const a = 1;

    const b = 2;

    "This Is A Bad Identification"; Expected an assignment or function call and instead saw an expression.

    if(a<b)
    {return;} Type 'undefined' is not assignable to type 'number'.

    const c = "This Variable Is Never Used"; 'c' is assigned a value but never used. Allowed unused vars must match /^_/u.

    getSystemErrorMap.console.log("You shold no use single quotes"); Cannot find name 'getSystemErrorMap'.
    expect(() => AwesomeFunc(invalidData)).toThrow(); Cannot find name 'expect'.
}

```

arreglo manual

```

TS example.ts > ...
1  function myAwesomeVariable() {
2
3      const a :number = 1;
4
5      const b =2;
6
7
8      if(a<b)
9      {
10         return a;
11         console.log("You shold no use single quotes");
12     }
13 }
14 myAwesomeVariable();
15

```

PS C:\Users\Wikor\Desktop\New folder (2)\project-backend> npx eslint .
 PS C:\Users\Wikor\Desktop\New folder (2)\project-backend>

No hay más errores

Entorno no controlado

The screenshot shows the VS Code interface with a file named `test.ts` open. The file contains Jest test code with several ESLint errors highlighted in red squiggly lines. The errors are related to indentation and trailing spaces. The bottom panel shows the 'PROBLEMS' view with 145 errors listed, and the status bar indicates 145 errors and 0 warnings.

```

test.ts test.ts > ...
1  import { describe, test, expect } from "vitest";
2  import { calcFinalOverTime, calcTimeInOven, calcHandlingTime, type SetData } from "../lib/portionCalcLogic";
3
4
5  describe("Pruebas unitarias: calcNumberOfSets", () => {
6      describe("Camino correcto", () => {
7          test("cálculo con unidades mayores a la capacidad de un set", () => {
8              expect(calcNumberOfSets(50, 10, 2)).toBe(3);
9          });
10         expect(calcNumberOfSets(50, 10, 2)).toBe(3);
11     });
12     describe("Casos Limite", () => {
13         test("Unidades exactamente en cero", () => {
14             expect(calcNumberOfSets(0, 10, 1)).toBe(0);
15         });
16         test("Unidades menores a cero", () => {
17             expect(calcNumberOfSets(-5, 10, 1)).toBe(0);
18         });
19         test("Unidades exactamente iguales a la capacidad del set", () => {
20             expect(calcNumberOfSets(10, 10, 1)).toBe(1);
21         });
22     });
23

```

PS C:\Users\Wikor\Desktop\New folder (2)\project-backend> npx eslint .
 171:11 error Expected indentation of 3 tabs but found 6 spaces indent
 172:11 error Expected indentation of 4 tabs but found 8 spaces indent
 173:11 error Expected indentation of 4 tabs but found 8 spaces indent
 174:11 error Expected indentation of 4 tabs but found 8 spaces indent
 175:11 error Expected indentation of 4 tabs but found 8 spaces indent
 176:11 error Expected indentation of 3 tabs but found 6 spaces indent
 177:11 error Expected indentation of 2 tabs but found 4 spaces indent
 178:11 error Expected indentation of 1 tab but found 2 spaces indent
 179:14 error Newline required at end of file but not found eol-last

X 145 problems (145 errors, 0 warnings)
 145 errors and 0 warnings potentially fixable with the '--fix' option.

PS C:\Users\Wikor\Desktop\New folder (2)\project-backend>

